REVERSE ENGINEERING // MASTERY COURSE

PHASE 3

Static Analysis

Bina chalaaye code padh ke samajhna — strings, functions, decompiler.

```
re@learning: ~/phase-3

$ ./start_learning.sh --phase 3
# Hinglish mein, simple examples ke saath — ek-ek step
$ status: READY ✓
```

🧠 Detailed Theory

💻 Code + Line-by-line

🎯 Interview Q&A

IN Hinglish

Phase 3 — Static Analysis (Bina Chalaaye Samajhna)

> **Goal:** Program ko **chalaye bina**, sirf code padh ke samajhna. Jaise book padhna bina act kiye. Ye RE ka core skill hai. Time: **3-4 hafte**.

3.0 — Static Analysis Kya Hai?

Static analysis matlab program ko run kiye bina analyze karna — bas uska code (assembly / decompiled C) padhkar samajhna ki wo kya karega.

Fayde: - ✅ Safe hai — malware bhi chalta nahi, isliye PC surakshit. - ✅ Poora code ek saath dikh jaata hai. - ✅ Chhupe paths bhi dikhte hain (jo shayad kabhi run na hon).

Nuksan: - ❌ Encrypted/packed code samajhna mushkil. - ❌ Runtime values (jaise decrypt hua password) nahi dikhte.

> Isiliye static + dynamic dono seekhte hain. Static se "kya karega" samajho, dynamic (Phase 4) se "kya kar raha hai" confirm karo.

3.1 — Step 1: File Ko Pehchano (Recon)

Analysis se pehle file ke baare mein basic info nikaalo:

1. **File type?** — Windows PE? Linux ELF? 32-bit ya 64-bit? (DIE tool se)
2. **Packed/encrypted?** — Agar packed hai toh pehle unpack karna padega (DIE batata hai).
3. **Compiler/language?** — C, C++, Go, Rust, .NET? (har ka reverse alag).
4. **Strings nikaalo** — sabse pehla clue yahan milta hai.

Strings — Sabse Powerful Pehla Kadam

```
strings program.exe           # sab readable text
strings program.exe | grep -i "pass"   # "password" jaisa dhoondo
```

Aksar milte hain: - Passwords, secret keys (hardcoded) - URLs, IP addresses (malware C2 servers) - Error/success messages ("Wrong password!", "Access granted!") - File paths, registry keys

> **Trick:** "Success" ya "Wrong" message dhoondo. Uske XREF se seedha password-check function mil jaata hai!

3.2 — Step 2: Entry Point Aur main() Dhoondo

Program kahan se shuru hota hai? - **Entry point** — technically program yahan se chalu hota hai (runtime setup code). - **main()** — asli logic yahan hota hai. Ghidra usually `main` dhoond ke label kar deta hai.

Ghidra mein: Symbol Tree → Functions → `main` (ya `entry`, `WinMain`). Wahan se logic follow karo.

3.3 — Step 3: Functions Ka Flow Follow Karo

Ek program bahut se functions ka jaal hota hai. Unhe samajhne ka tarika:

1. `main` se shuru karo.
2. Dekho `main` kaunse functions ko `call` karta hai.
3. Har important function ko rename karo (jaise `FUN_00401000` → `check_password`).
4. Chhote helper functions pehle samjho, phir bade.

Call Graph

Ghidra ek "call graph" bana sakta hai — dikhata hai kaunsa function kise call karta hai. Isse poore program ka nakshaa mil jaata hai.

3.4 — Step 4: Control Flow Samajhna (If, Loops)

Decompiled/assembly code mein logic ko pehchano:

If-else pehchano

```
// Decompiled Ghidra output aisa dikhta hai:
if (iVar1 == 0x2a) {          // agar iVar1 == 42
    puts("Correct!");
} else {
    puts("Wrong!");
}
```

Assembly mein yahi `cmp` + `je` / `jne` hota hai (Phase 1 wala).

Loops pehchano

```
for (i = 0; i < 10; i = i + 1) {
    sum = sum + i;
}
```

Assembly mein loop ek "backward jump" se banta hai (upar wali line pe wapas kudna).

Control Flow Graph (CFG)

Ghidra CFG banata hai — boxes (basic blocks) aur arrows (jumps). Isse dekho code kaise flow karta hai — kaunsi condition kahan le jaati hai.

3.5 — Step 5: Decompiler Ka Sahi Use

Ghidra ka decompiler assembly ko C-jaisa dikhata hai. Ye padhna aasan hai, but kuch baatein yaad rakho:

- Variable naam ajeeb honge: `local_8`, `iVar1`, `uVar2`, `param_1`. Inhe **rename** karo jaise samajhte jao.
- Types galat ho sakte hain (int vs pointer). Ghidra ko sahi type batao (`Ctrl+L`).
- Kabhi decompiler galti karta hai — doubt ho toh assembly (Listing window) dekho.

Common function names pehchano

Ye standard library functions dikhein toh turant matlab samjho: | Function | Matlab | |----------|--------| | `strcmp` / `strncmp` | do strings compare (aksar password check!) | | `strcpy` / `memcpy` | data copy | | `printf` / `puts` | screen pe print | | `scanf` / `gets` / `fgets` | user se input lena | | `malloc` / `free` | memory allocate/release | | `fopen` / `fread` | file operations |

> `strcmp` dikha? 90% chance wahan koi string comparison ho raha hai — password ya key check.

3.6 — Real Example: Ek Password Checker Crack Karna

Chalo ek typical "crackme" static analysis se todte hain.

Program ka behaviour

```
$ ./crackme
Enter password: hello
Wrong password!
```

Ghidra decompiler output (typical)

```
int main(void) {
    char input[32];              // user ka input yahan aayega
    printf("Enter password: ");
    scanf("%s", input);          // user se password liya
    if (strcmp(input, "Sup3rS3cr3t") == 0) {   // compare!
        puts("Access granted!");
        return 0;
    }
    puts("Wrong password!");
    return 1;
}
```

Line by line samjho: - `char input[32];` — 32 characters ka box, user input ke liye. - `scanf("%s", input);` — user se password padha, `input` mein daala. - `strcmp(input, "Sup3rS3cr3t")` — user ka input "Sup3rS3cr3t" se compare kiya. `== 0` matlab dono barabar (strcmp equal pe 0 deta hai). - Agar barabar → "Access granted!", warna "Wrong password!".

> **Password mil gaya:** `Sup3rS3cr3t` — bina program chalaye, sirf code padh ke! Ye static analysis ki power hai.

Thoda mushkil version (obfuscated)

Kabhi password seedha nahi hota — thoda "encode" hota hai:

```
// har character ko XOR kiya gaya 0x10 se
char secret[] = {0x63, 0x75, 0x64, ...};   // encoded bytes
for (i = 0; i < len; i++) {
    if ((input[i] ^ 0x10) != secret[i]) {   // input ko bhi XOR karke check
        // wrong
    }
}
```

Yahan tumhe khud `secret` bytes ko `0x10` se XOR karke asli password nikaalna padega. (Ya dynamic analysis se dekh lo — Phase 4.)

3.7 — Static Analysis Workflow (Summary)

```
1. Recon    → DIE se file type, packed check
2. Strings  → clues (passwords, messages, URLs)
3. Entry    → main() dhoondo
4. Flow     → functions follow karo, rename karo
5. Logic    → if/loops/strcmp pehchano
6. Solve    → password/key/logic nikaalo
7. Notes    → comments aur renames karte raho
```

🧪 Phase 3 — Hands-On Practice

- crackmes.one se **easy** password-based crackmes lo.
- Sirf static analysis (Ghidra) se solve karne ki koshish karo.
- Har function ko rename + comment karo.
- Kam se kam 3-4 crackmes static se solve karo.

🎯 Key Takeaways (Ek Line Mein)

1. Static analysis = program **run kiye bina** code padh ke samajhna (safe + complete view).
2. **Strings pehla step** — passwords, messages, URLs aksar seedha mil jaate hain.
3. `main()` se shuru karo, functions ko follow aur rename karte jao.
4. `cmp` +jump = if-else; backward jump = loop.
5. `strcmp` dikha = string comparison (aksar password check).
6. Decompiler aasan hai but approximate — doubt pe assembly dekho.
7. Encrypted password ko XOR/decode karke nikaalna pad sakta hai.

> ✅ **Phase 3 Complete Jab:** Tum ek simple program ka decompiled code padh ke samajh sako ki wo kya logic use kar raha, aur ek easy crackme static analysis se solve kar sako. Ab dynamic

> analysis (Phase 4)! 🚀

Phase 3 — Interview Questions & Answers (Hinglish)

> Static Analysis se related important interview questions, simple Hinglish jawaab ke saath.

Q1. Static analysis kya hai?

Ans: Static analysis matlab program ko **chalaye bina** analyze karna — sirf uska code (assembly ya decompiled C) padhkar samajhna ki wo kya karega. Ye safe hota hai (malware bhi chalta nahi) aur poora code ek saath dikh jaata hai.

Q2. Static analysis ke fayde aur nuksan kya hain?

Ans: - **Fayde:** Safe hai (kuch execute nahi hota), poora code aur saare code paths dikhte hain (chhupe hue bhi). - **Nuksan:** Encrypted/packed code samajhna mushkil, aur runtime pe banne wali values (jaise decrypt hua password) nahi dikhtin.

Isliye ise dynamic analysis ke saath milaake use karte hain.

Q3. Kisi program ka analysis shuru karte waqt pehla step kya hota hai?

Ans: File ka **recon** — file type (PE/ELF, 32/64-bit), packed hai ya nahi (DIE tool se), aur kaunse compiler/language se bani. Uske baad **strings** nikaalna, jahan aksar passwords, messages, aur URLs jaise clues seedha mil jaate hain.

Q4. Strings analysis se kya milta hai?

Ans: File ke andar chhupe readable text — jaise hardcoded passwords/keys, URLs ya IP addresses (malware ke C2 servers), success/error messages ("Access granted!", "Wrong password!"), file paths, aur registry keys. Ye pehla aur sabse aasan clue source hai.

Q5. Entry point aur main() mein kya farak hai?

Ans: Entry point wo jagah hai jahan se program technically chalu hota hai — ismein runtime setup code hota hai. **main()** wo function hai jahan program ka asli logic likha hota hai. RE mein hum usually main() se analysis shuru karte hain.

Q6. Program ka control flow static analysis mein kaise pehchante hain?

Ans: If-else `cmp` + conditional jump (`je` / `jne`) se bante hain, aur loops ek backward jump (upar wali line pe wapas kudna) se. Decompiler ismein `if`, `for`, `while` jaisa dikha deta hai. Ghidra ka Control Flow Graph (CFG) boxes aur arrows se ye flow visually dikhata hai.

Q7. `strcmp` function dikhe toh kya samajhna chahiye?

Ans: `strcmp` do strings ko compare karta hai aur barabar hone pe **0** return karta hai. RE mein jab `strcmp(input, "kuch") == 0` dikhe, to samajh jao wahan koi string comparison ho raha hai — aksar password ya key check. Ye function seedha logic tak le jaata hai.

Q8. Decompiler output padhte waqt kaunsi baatein dhyaan mein rakhni chahiye?

Ans: Variable naam ajeeb honge (`local_8`, `iVar1`, `param_1`) — unhe samajhte hue rename karo. Types galat ho sakte hain (int vs pointer) — sahi type set karo. Aur decompiler kabhi galti karta hai — doubt ho to assembly (Listing) dekho.

Q9. Rename aur comment karna static analysis mein kyun zaroori hai?

Ans: Decompiler generic naam deta hai jo samajhna mushkil hai. Jaise-jaise logic samajhte ho, functions/variables ko meaningful naam do aur notes (comments) likho. Ye ek "map banane" jaisa hai — jitna map banega, poora program utna clear hota jaayega.

Q10. Agar password seedha strcmp mein na ho, encoded ho, tab kya karoge?

Ans: Aksar password XOR ya kisi simple encoding se chhupaya hota hai. Tab code se encoding logic samajhkar (jaise har byte ko `0x10` se XOR) usko ulta karke asli password nikaalna padta hai. Ya dynamic analysis mein breakpoint lagaakar decrypt hui value memory mein dekh lete hain.

Q11. Cross-references (XREF) static analysis mein kaise madad karte hain?

Ans: XREF batata hai koi string ya function kahan use hua. Jaise agar "Wrong password!" string mili, uske XREF se seedha wo function mil jaata hai jahan password check ho raha hai. Isse important logic tak jaldi pahunch jaate hain bina poora code padhe.

Q12. Packed binary static analysis mein problem kyun banta hai?

Ans: Packed binary ka asli code compress/encrypt hota hai, isliye static mein sirf packer ka code aur garbage dikhta hai — asli logic aur strings chhupe hote hain. Ise pehle unpack karna padta hai (ya dynamic analysis mein memory se dump karna padta hai) tab hi asli code analyze ho paata hai.

Phase 3 — Interview Questions & Answers (English)

> The same Static Analysis interview questions, answered in clear English for formal interviews.

Q1. What is static analysis?

Ans: Static analysis means analyzing a program **without running it** — understanding what it will do by reading its code (assembly or decompiled C). It's safe (even malware never executes) and shows the whole code at once.

Q2. What are the advantages and disadvantages of static analysis?

Ans: - **Advantages:** It's safe (nothing executes), and you see the entire code and all code paths (even hidden ones). - **Disadvantages:** Encrypted/packed code is hard to read, and runtime-generated values (like a decrypted password) aren't visible.

That's why it's combined with dynamic analysis.

Q3. What is the first step when starting to analyze a program?

Ans: File **recon** — identify the file type (PE/ELF, 32/64-bit), whether it's packed (using a tool like DIE), and which compiler/language produced it. Then extract **strings**, which often directly reveal clues like passwords, messages, and URLs.

Q4. What do you get from strings analysis?

Ans: Readable text hidden in the file — hardcoded passwords/keys, URLs or IP addresses (malware C2 servers), success/error messages ("Access granted!", "Wrong password!"), file paths, and registry keys. It's the first and easiest source of clues.

Q5. What is the difference between the entry point and main()?

Ans: The **entry point** is where the program technically begins — it contains runtime setup code. **main()** is the function containing the program's actual logic. In RE we usually start analysis from main().

Q6. How do you recognize control flow in static analysis?

Ans: If-else is formed by `cmp` plus a conditional jump (`je` / `jne`), and loops by a backward jump (jumping back to an earlier line). The decompiler shows these as `if` , `for` , `while` . Ghidra's Control Flow Graph (CFG) displays this flow visually as boxes and arrows.

Q7. What should you infer when you see the `strcmp` function?

Ans: `strcmp` compares two strings and returns **0** when they're equal. In RE, seeing `strcmp(input, "something") == 0` signals a string comparison — often a password or key check. It leads you straight to the important logic.

Q8. What should you keep in mind while reading decompiler output?

Ans: Variable names will be generic (`local_8` , `iVar1` , `param_1`) — rename them as you understand them. Types may be wrong (int vs pointer) — set the correct type. And the decompiler occasionally makes mistakes — when in doubt, check the assembly (Listing view).

Q9. Why are renaming and commenting important in static analysis?

Ans: The decompiler assigns generic names that are hard to follow. As you understand the logic, give functions/variables meaningful names and add notes (comments). It's like drawing a map — the more you map, the clearer the whole program becomes.

Q10. What do you do if the password isn't in a plain strcmp but is encoded?

Ans: Passwords are often hidden with XOR or simple encoding. You then read the encoding logic (e.g., each byte XOR-ed with `0x10`) and reverse it to recover the real password. Alternatively, in dynamic

analysis, set a breakpoint and read the decrypted value directly from memory.

Q11. How do cross-references (XREFs) help in static analysis?

Ans: XREFs show where a string or function is used. For example, if you find the "Wrong password!" string, its XREF leads directly to the function performing the password check. This gets you to the important logic quickly without reading all the code.

Q12. Why is a packed binary a problem for static analysis?

Ans: A packed binary's real code is compressed/encrypted, so static analysis shows only the packer's stub and garbage — the real logic and strings are hidden. It must first be unpacked (or dumped from memory during dynamic analysis) before the real code can be analyzed.